QUESTION 1:

Every state is either a P or N position.
P -> previous player wins
N-> next player wins

Legal move -> 1,3,4
Game ends when pile empty

N = 0: no stones left meaning the previous player has the final stone so they won P
N = 1: one stone left, so we take it and win N
N =2: two stones left, 2 is not a valid move so we take 1, opponent takes 1 and wins P
N =3: three stones left, you take 3 and win N
N = 4: four stones left, take 4, win N
N = 5: five stones left, take 3 to leave 2 (P) and win N
N = 6: six stones left, take 4 to leave 2 (P) and win N
N = 7: seven stones left, taking 1, 3, or 4 leaves 6, 4, or 3 (N positions) so opponent win P
N = 8: eight stones left, take 1 to leave 7 (P) and win N
N = 9: nine stones left, taking 1, 3, or 4 leaves 8, 6, or 5 (N positions) so opponent win P
N = 10: ten stones left, take 1 to leave 9 (P), and win N
N = 11: eleven stones left, take 4 to leave 7 (P),and win N
N = 12: twelve stones left, take 3 to leave 9 (P) and win N

P Positions:
0, 2, 7, 9

N = 21 and 28 are both P positions for the first player. Reason: the p positions sequence is switching between n mod 7 = 0 and n mod 7 = 2. 21 mod 7 =0 and 28 mod 7 = 0 also

If we can move 1, 2, or 3 stones the pattern becomes 0, 4, 8, 12 and so on making it repeat every 4 stones so if the total number of stones removed every single round is 4, it is a guaranteed win. For the current structure, since 2 is missing we cannot make the total removed 4 if the opp chooses 3 or 4 since there is no 0 and 2, hence the pattern ends up repeating at 7 stones which is harder to keep track of.

For N = 8, Minimax will search 64 nodes (1,2,3,5,9,15,24…..)
With Aplha beta pruninh, it will search either around 15 to 30 nodes. This is due to what moves are played affecting the game with pruning called Move Ordering.

Implementation Screenshots:

```
--- Diagnostic: Algorithm Efficiency ---
Calculating tree for N=15 without pruning...
Calculating tree for N=15 with Alpha-Beta pruning...
Nodes evaluated (Naive): 1869
Nodes evaluated (Alpha-Beta): 149
Pruning reduced computational load by: 92.03%
----------------------------------------

Starting Game! Initial stone count: 27

[ STONES REMAINING: 27 ]
Your turn. Remove 1, 3, or 4 stones: 1
> You took 1 stones.

AI is calculating its move...
> AI takes 3 stones.

[ STONES REMAINING: 23 ]
Your turn. Remove 1, 3, or 4 stones: 2
Invalid input. You can only remove 1, 3, or 4 stones.
Your turn. Remove 1, 3, or 4 stones: 3
> You took 3 stones.

AI is calculating its move...
> AI takes 4 stones.

[ STONES REMAINING: 16 ]
Your turn. Remove 1, 3, or 4 stones: 5
Invalid input. You can only remove 1, 3, or 4 stones.
Your turn. Remove 1, 3, or 4 stones: -1
Invalid input. You can only remove 1, 3, or 4 stones.
Your turn. Remove 1, 3, or 4 stones: 0
Invalid input. You can only remove 1, 3, or 4 stones.
Your turn. Remove 1, 3, or 4 stones: SDS
Invalid input. Please enter a valid integer.
Your turn. Remove 1, 3, or 4 stones: 1
> You took 1 stones.

AI is calculating its move...
> AI takes 1 stones.

[ STONES REMAINING: 14 ]
Your turn. Remove 1, 3, or 4 stones: 2
Invalid input. You can only remove 1, 3, or 4 stones.
Your turn. Remove 1, 3, or 4 stones: 4
> You took 4 stones.

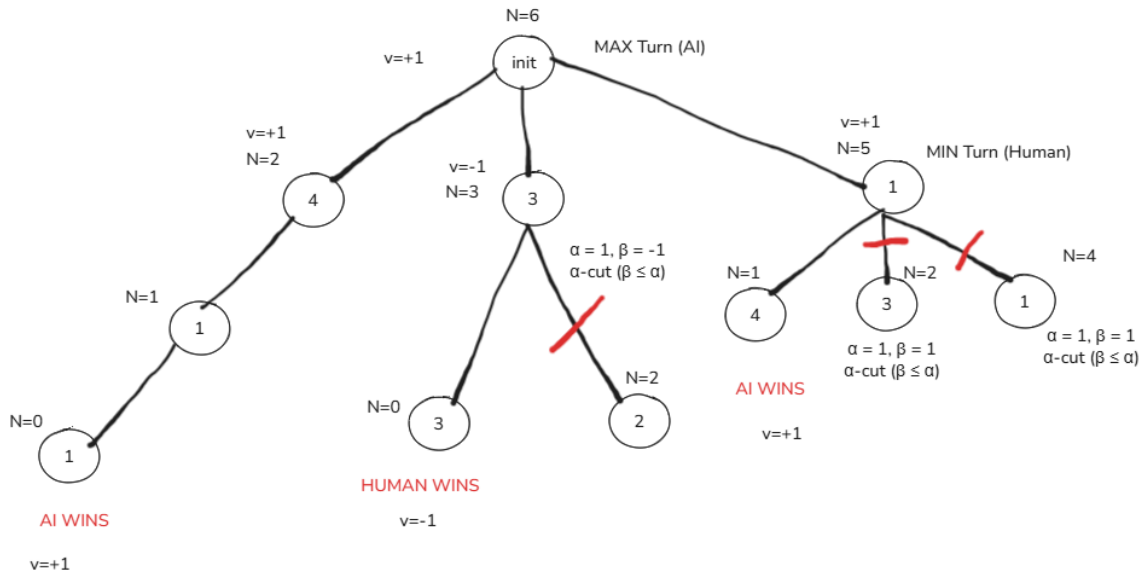
AI is calculating its move...
> AI takes 3 stones.

[ STONES REMAINING: 7 ]
Your turn. Remove 1, 3, or 4 stones: 1
> You took 1 stones.

AI is calculating its move...
> AI takes 4 stones.

[ STONES REMAINING: 2 ]
```

Game Tree Trace:

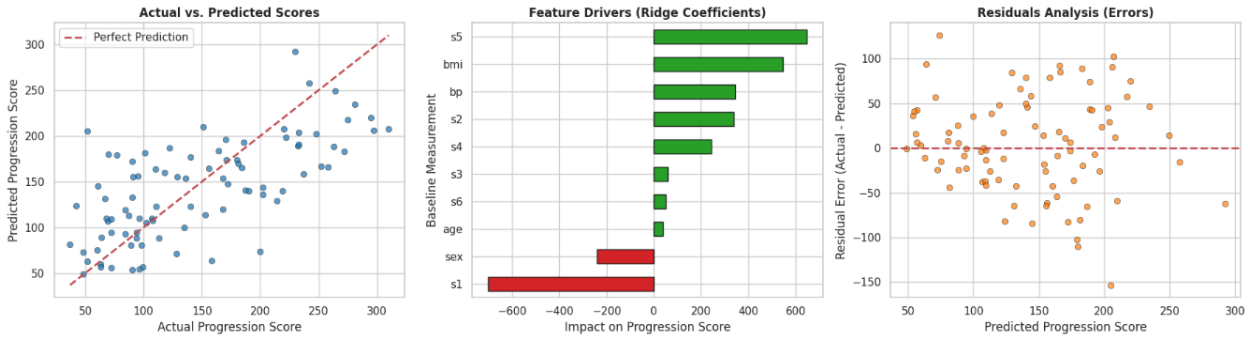


Critical reflection:
If the opponent plays poorly, the algorithm will recalculate the game tree from the weaker position and win. It never plays better than necessary because it considers a slow win the same as a quick win. To punish mistakes, we would have to change it so that it is punished for going into a deeper branch. The algorithm assumes perfection and because of that it assumes perfection from the opponent as well.

A game with many possible moves would make the algorithm computationally infeasible. Example: Chess has around 30 moves at any given time on average. An additional required technique would be depth limiting since it is too computationally intensive to search the entire branch at a time.

A lookup table requires someone to sit down and do the max and find the pattern and then hardcode the solution. Minimax doesnt need to understand the pattern it only needs the rules, it is not hardcoded and discovers the optimal path entirely by itself. Minimax is also able to adapt if you decide to change the game immediately. Also, you cannot expect someone to do the math for finding the lookup table for a game like chess. Minimax can do it much easier.

QUESTION 2:
Client Request 1:



```
--- Data Snapshot ---
        age       sex       bmi        bp        s1        s2        s3  \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356

         s4        s5        s6
0 -0.002592  0.019907 -0.017646
1 -0.039493 -0.068332 -0.092204
2 -0.002592  0.002861 -0.025930

Target Sample (Numerical): [151.  75. 141.]

--- Model Evaluation ---
Optimal Regularization Penalty (Alpha): 0.0027
Mean Absolute Error (MAE): 42.81
Root Mean Squared Error (RMSE): 53.77
R-squared (R2): 0.4543
```

Since the target is a numerical value, we use a regression model. Specifically, Ridge Regression is being used since the features are massively correlated. MAE is the main metric because its easy to interpret by looking at the graph and its distance based so its easy to understand (far from line = bad)

Reasoning part:
We turned the regression problem into binary classification. The distance based metrics like mae and R2 and invalid now because all of the points will lie either on 1 or 0 which will destroy the distance metrics. We need classification metrics now like Confusion matrix and roc-auc curve which would be able to identify correct and incorrect predictions

Client Request 2:

```
--- Data Reality Check ---
Class Distribution:
Benign       357
Malignant    212
Name: count, dtype: int64

Notice the imbalance. If we just guess 'Benign', we are 63% accurate, but we miss every cancer case.

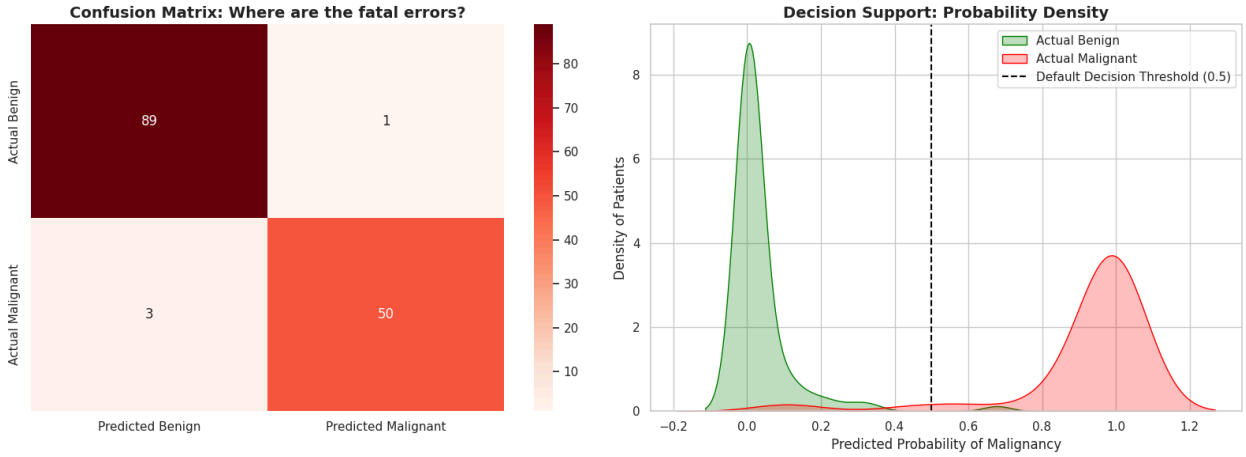
--- Hard Decision vs Probability Example (First 3 Patients) ---
Patient 1: Hard Decision = Malignant (1) | Model Confidence (Probability of Malignancy) = 0.9997
Patient 2: Hard Decision = Benign (0) | Model Confidence (Probability of Malignancy) = 0.0052
Patient 3: Hard Decision = Malignant (1) | Model Confidence (Probability of Malignancy) = 0.9998

--- Evaluation (Focusing on Medical Risk) ---
ROC-AUC Score: 0.9950 (Shows ability to rank patients by risk)
RECALL (Sensitivity) for Malignant: 0.9434 -> This means we caught 94.34% of actual cancer cases.

Full Classification Report:
              precision    recall  f1-score   support

      Benign       0.97      0.99      0.98        90
   Malignant       0.98      0.94      0.96        53

    accuracy                           0.97       143
   macro avg       0.97      0.97      0.97       143
weighted avg       0.97      0.97      0.97       143
```



The client needs a diagnosis (binary) with a confidence score so we can use a classification model. Specifically, logistic regression because it naturally outputs probabilities instead of outputting only categories. Recall is our main metric because we want to minimize false negatives and not miss dangerous cases (miss cancer = patient dies)

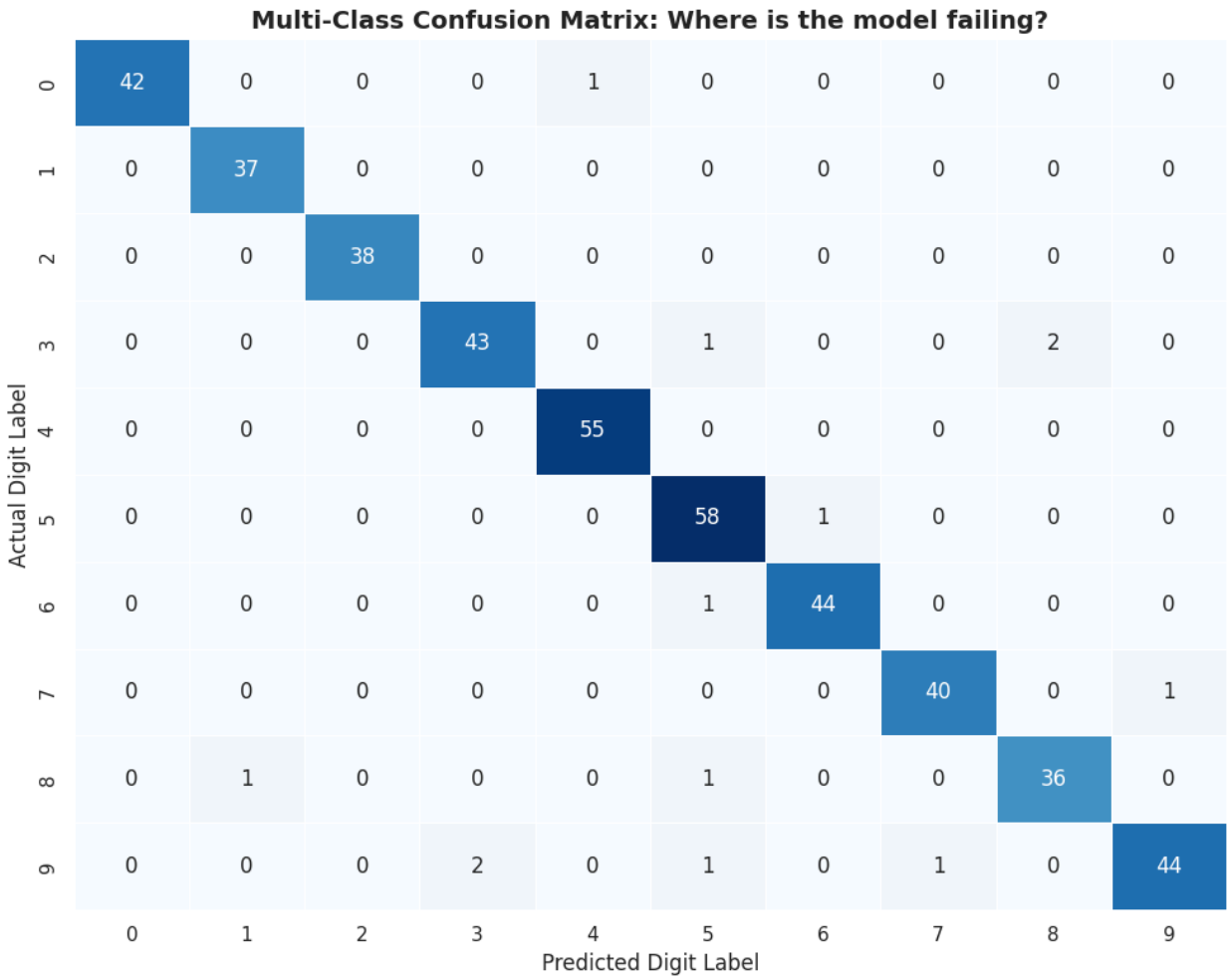
Reasoning part:
The changed rule adjusts the precision-recall tradeoff. By lowering the decision threshold, recall increases (more flagged as malignant, so precision decreases) and by increasing the decision threshold, the precision increases(more not flagged as malignant, so recall decreases). The safer rule is 0.30 because we do not want to miss a dangerous case.

Client Request 3:

```
--- Detailed Evaluation (Class-Level & Macro) ---
              precision    recall  f1-score   support

           0       1.00      0.98      0.99        43
           1       0.97      1.00      0.99        37
           2       1.00      1.00      1.00        38
           3       0.96      0.93      0.95        46
           4       0.98      1.00      0.99        55
           5       0.94      0.98      0.96        59
           6       0.98      0.98      0.98        45
           7       0.98      0.98      0.98        41
           8       0.95      0.95      0.95        38
           9       0.98      0.92      0.95        48

    accuracy                           0.97       450
   macro avg       0.97      0.97      0.97       450
weighted avg       0.97      0.97      0.97       450
```



The client requires a multi-class classification so we use random forest since it handles multiple class boundaries. A confusion matrix and classification report are our main metrics because we want to expose class-level errors and also avoid hiding failures behind the overall accuracy.

Reasoning part:
Random forest is just multiple decision trees so we cannot apply an iteration limit. They use rules not weights being optimized so there is no convergence process.

Convergence means the optimal weights have been found and early stopping with low iterations can cause those optimal weights to not be found. Convergence alone does not guarantee accuracy because optimal weights doesnt mean the model generalized, it just means it minimized the function it was working on and could have just memorized noise.

Table:

Client Request	Target Type	Model	Main Metrics	Reason for choice
Score Estimation	Regression	Ridge Regression	MAE, R^2	Ridge penalizes coefficients, standard regression overfits to noise
Two-outcome screening	Binary Classification	Logistic Regression	Recall, ROC-AUC	Naturally outputs confidence probabilities instead of a category
Many label id	Multiclass Classification	Random Forest	Macro-average and confusion matrix	Natively handles multiple non linear category boundaries